Excepciones en PL/SQL

Manejo de errores en la base de datos

Bases de Datos · Ingeniería de Sistemas · Oracle Database 19c

Excepciones

Bases de Datos · Ingeniería de Sistemas · PL/SQL

Dónde estamos

- Ya sabemos escribir procedimientos y funciones que hacen lo correcto cuando todo sale bien. Falta la otra mitad: qué ocurre cuando algo falla.
- El error no es la excepción: el error es no atenderla, o peor, esconderla.

Recorrido

- Qué es una excepción y cómo altera el flujo del bloque
- Las tres familias: predefinidas, errores ORA nombrados y propias del negocio
- RAISE, RAISE_APPLICATION_ERROR y PRAGMA EXCEPTION_INIT
- WHEN OTHERS, propagación, transacciones y antipatrones

Qué es una excepción

Una condición que interrumpe la ejecución normal

- Cuando se levanta una excepción, la ejecución abandona el cuerpo del bloque y salta a la sección EXCEPTION.
- Si el bloque la maneja, el programa continúa después de END.
- Si no la maneja, la excepción se propaga al bloque que lo llamó.
- Si nadie la maneja, llega al cliente como un error ORA y la transacción queda a medias.

Una excepción no es un fallo del programa: es información sobre lo que ocurrió.

Anatomía del bloque


```
DECLARE
    -- declaraciones: variables, cursores, excepciones propias
BEGIN
    -- cuerpo: aqui se levantan las excepciones
EXCEPTION
    WHEN excepcion_1 THEN
        -- tratamiento especifico
    WHEN excepcion_2 THEN
        -- tratamiento especifico
    WHEN OTHERS THEN
        -- ultimo recurso: registrar y relanzar
END;
```

Un error dentro de la sección EXCEPTION no lo atiende ese mismo bloque: se propaga hacia afuera.

Las tres familias de excepciones

Familia	Tiene nombre	Cómo se declara	Ejemplo
Predefinidas	Sí	Ya existen en el lenguaje	NO_DATA_FOUND
No predefinidas	No	PRAGMA EXCEPTION_INIT	ORA-02291
Definidas por el usuario	Sí	Declaración EXCEPTION	e_stock_insuficiente

- Las predefinidas cubren los errores más comunes del motor.
- Las no predefinidas son errores ORA a los que Oracle no asignó nombre.
- Las del usuario representan reglas de negocio, no fallas técnicas.

Excepciones predefinidas más frecuentes

Excepción	ORA	Cuándo se levanta
NO_DATA_FOUND	01403	SELECT ... INTO no devolvió ninguna fila
TOO_MANY_ROWS	01422	SELECT ... INTO devolvió más de una fila
DUP_VAL_ON_INDEX	00001	Se violó una restricción única
ZERO_DIVIDE	01476	División por cero
INVALID_NUMBER	01722	Conversión fallida de cadena a número
VALUE_ERROR	06502	Error aritmético o de truncamiento en la asignación
CURSOR_ALREADY_OPEN	06511	OPEN sobre un cursor ya abierto

NO_DATA_FOUND la levanta SELECT ... INTO, nunca un FETCH sobre cursor explícito: allí se usa %NOTFOUND.

Excepciones definidas por el usuario

Nombrar la regla de negocio en lugar de esconderla dentro de un condicional.

```
CREATE OR REPLACE PROCEDURE sp_validar_despacho (
    p_id_producto NUMBER, p_id_bodega NUMBER, p_cantidad NUMBER
) IS
    e_stock_insuficiente EXCEPTION;
    v_disponible NUMBER;
BEGIN
    v_disponible := fn_stock_disponible(p_id_producto, p_id_bodega);

    IF v_disponible < p_cantidad THEN
        RAISE e_stock_insuficiente;
    END IF;
EXCEPTION
    WHEN e_stock_insuficiente THEN
        DBMS_OUTPUT.PUT_LINE('Disponible: ' || v_disponible);
END;
```

RAISE_APPLICATION_ERROR

Devolver un error propio, con número y mensaje, hacia la aplicación cliente.

```
IF v_estado = 'ANULADO' THEN
    RAISE_APPLICATION_ERROR(-20011,
        'El pedido ' || p_id_pedido || ' ya se encuentra anulado.');
END IF;
```

- El rango permitido va de -20000 a -20999.
- El mensaje admite hasta 2.048 bytes.
- El cliente lo recibe como un ORA- estándar y puede distinguirlo por el número.
- Conviene mantener un catálogo de códigos documentado por módulo.

RAISE o RAISE_APPLICATION_ERROR

RAISE

- Levanta una excepción declarada
- Visible solo dentro de PL/SQL
- Si escapa, el cliente recibe ORA-06510
- Para control de flujo interno

RAISE_APPLICATION_ERROR

- Genera un error con número propio
- Viaja hasta la aplicación cliente
- El cliente lo distingue por su código
- Para comunicar reglas de negocio

Un procedimiento que valida reglas de negocio y solo usa RAISE deja al cliente sin forma de saber qué regla se incumplió.

PRAGMA EXCEPTION_INIT

Ponerle nombre a un error del motor para poder atenderlo por separado.

```
DECLARE
    e_fk_violada EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_fk_violada, -2291);
BEGIN
    INSERT INTO producto (id_producto, codigo, id_categoria, ...)
    VALUES (seq_producto.NEXTVAL, 'XX-9999', 99, ...);
EXCEPTION
    WHEN e_fk_violada THEN
        DBMS_OUTPUT.PUT_LINE('La categoria indicada no existe.');
END;
```

Permite atender un error concreto sin recurrir a WHEN OTHERS: los demás errores siguen propagándose.

SQLCODE, SQLERRM y el rastro del error

Función	Qué devuelve
SQLCODE	Número del error en curso; 0 si no hay error, 1 si es del usuario
SQLERRM	Mensaje asociado al error en curso
DBMS_UTILITY.FORMAT_ERROR_STACK	Pila de errores completa
DBMS_UTILITY.FORMAT_ERROR_BACKTRACE	Línea exacta donde se originó el error

SQLERRM dice qué pasó; FORMAT_ERROR_BACKTRACE dice dónde. Registrar solo el primero obliga a adivinar la línea.

WHEN OTHERS

Bases de Datos · Ingeniería de Sistemas · PL/SQL

El manejador más usado y el peor usado.

```
-- ANTIPATRON: el error desaparece
EXCEPTION
    WHEN OTHERS THEN
        NULL;

-- CORRECTO: registrar y relanzar
EXCEPTION
    WHEN OTHERS THEN
        sp_registrar_error('sp_facturar', SQLERRM);
        RAISE;
```

WHEN OTHERS THEN NULL convierte un fallo en un silencio: es la causa de que nadie se entere cuando la facturación no ocurre.

Propagación entre bloques

- Si el bloque no maneja la excepción, esta sube al bloque que lo llamó.
- En cada nivel se busca un manejador que coincida; el primero que aplique la atiende.
- Manejarla y no relanzarla equivale a declarar que el problema quedó resuelto.

```
BEGIN
    BEGIN
        SELECT saldo INTO v_s FROM cliente WHERE id_cliente = p_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN v_s := 0;   -- caso previsto
    END;
    -- el resto de errores sigue subiendo
END;
```

Un bloque interno puede atender el caso local y dejar pasar todo lo demás. Es la forma correcta de acotar el alcance.

Excepciones y transacciones

Una excepción no revierte nada por sí sola

- Cuando se levanta una excepción, el trabajo hecho antes sigue pendiente en la transacción.
- Solo un ROLLBACK explícito, o la desconexión, lo revierte.
- La sentencia que falla sí se deshace por completo: es atómica a nivel de sentencia.
- SAVEPOINT permite revertir hasta un punto sin perder toda la transacción.

```
SAVEPOINT sp_inicio;
...
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK TO sp_inicio;
        RAISE;
```

Registrar el error aunque todo se revierta

Bases de Datos · Ingeniería de Sistemas · PL/SQL

Transacción autónoma para la bitácora de errores.

```
CREATE OR REPLACE PROCEDURE sp_registrar_error (
    p_origen VARCHAR2, p_detalle VARCHAR2 DEFAULT NULL
) IS
    PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    INSERT INTO log_errores (id_log, origen, codigo_error, mensaje)
    VALUES (seq_log.NEXTVAL, p_origen, SQLCODE, SQLERRM);
    COMMIT;
END;
```

Sin la transacción autónoma, el ROLLBACK que revierte el fallo borra también la evidencia del fallo. El COMMIT interno es obligatorio.

Antipatrones

Bases de Datos · Ingeniería de Sistemas · PL/SQL

Antipatrón	Por qué es un problema
WHEN OTHERS THEN NULL	El fallo desaparece sin rastro y el proceso parece exitoso
WHEN OTHERS como único manejador	Trata igual un dato faltante que una caída de red
Capturar y no relanzar ni registrar	El llamador cree que la operación se completó
Usar excepciones como control de flujo normal	Encarece la ejecución y confunde la lectura
Mensajes genéricos tipo 'Error en el proceso'	Obliga a depurar desde cero cada incidente
Códigos fuera del rango -20000 a -20999	ORA-21000: el número no es válido

Buenas prácticas

- Atienda primero lo específico y deje WHEN OTHERS como último recurso.
- Registre siempre SQLCODE, SQLERRM y el backtrace antes de relanzar.
- Nombre las condiciones de negocio con excepciones propias, no con condicionales anónimos.
- Mantenga un catálogo de códigos -20xxx documentado por módulo.
- Decida explícitamente el alcance transaccional: qué se revierte y hasta dónde.
- No haga COMMIT dentro de un procedimiento que otro pueda invocar.

Para retener

- La excepción informa; el error real es ocultarla.
- Tres familias: predefinidas, errores ORA nombrados con PRAGMA, y las propias del negocio.
- RAISE se queda en PL/SQL; RAISE_APPLICATION_ERROR llega al cliente.
- Una excepción no revierte la transacción: eso lo decide usted con ROLLBACK o SAVEPOINT.
- La transacción autónoma es la que salva la evidencia cuando todo lo demás se revierte.

En el taller: la Parte 4 recorre los cinco mecanismos, y el reto final se apoya en todos ellos.